Page No
Date / /

Lecture 22

Pointer

int * p;

p is a pointer which point a integer type of value.

int num = 5;

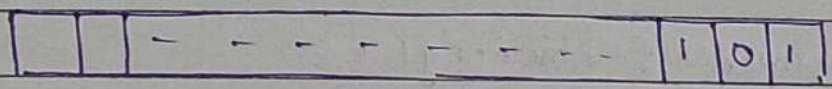
↓

4 byte = 32 bit.

Byte addressable : Every byte has assigned with a address.

int num = 5 → 101

4 byte = 32 bit.



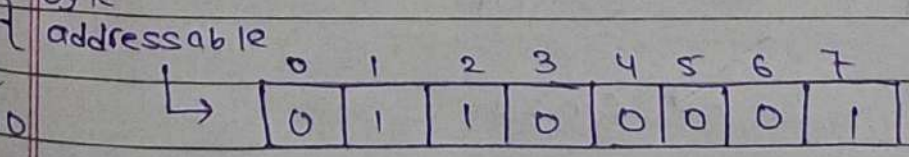
↳ 5 is stored like this in memory.

char c = 'a'; → ASCII → 97 → Binary ↴

1100001

1 byte : 8 bit

Byte addressable



200

↳ 'a' is stored like this in memory.

It means it have to read 8 bit because it is Byte addressable.

Teacher's Signature.....

int num = 5

address

p

p = #

p = 200 → address of num

* p = 20 ;

↳ it change the value of variable whose address stored in p.

200

200 | num | 20

p

Now, num = 20.

*p → ye apne adress pe jisko bhi store karwaya hua hai uske stath change kar sakta hai.

pointer : memory

address

4 GB Ram : 2^{32} byte.

pointer : → 32 bit

→ 64 bit

Teacher's Signature.....

0000 - - - - 0000

0000 - - - - 0001

0000 - - - 0010

⋮

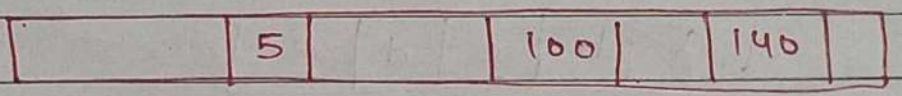
$2^{32} - 1$

0 to $2^{32} - 1$

Symbol table :

Variable	Address	Type

Num = 5 int * p = #



iska bhi
Kuch address
hoga.
(140)

int *p;

↳ p can contain address.

It store garbage value if not any value stored.

We put for clarification

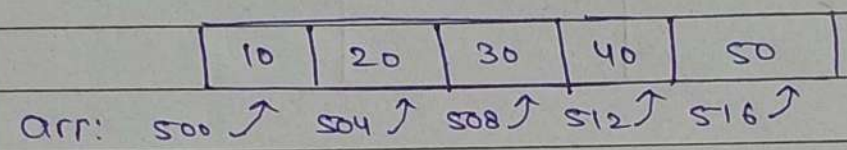
Put → NULL; or 0;

Teacher's Signature.....

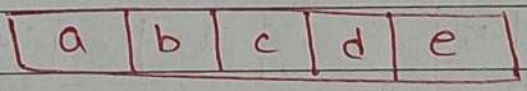
```
int num = 30
int * p;
p = & num;
* p = 20;
cout << num;
```

```
int arr [5] = { 10, 20, 30, 40, 50 }
```

20 byte : 4 byte each



Continous




```
arr[0] = arr + 0
arr[1] = arr + 1
arr[2] = arr + 2
```

→
```
int arr[5] = { 10, 20, 30, 40, 50 };
cout << arr ; // print address
    ↳ print in Hexadecimal
      Form.
cout << * arr;
    ↳ print arr[0];
```

Teacher's Signature.....

```
cout << *(arr+0);
        ↳ print arr[0];
cout << *(arr+1);
        ↳ print arr[1];

p = arr;
  ↳ it is already a adress
```

Eg, `int num = 5;`

```
int *p;
p = &num;
*p;
*p(+1);
*(p+2);

cout << *p;
    ↳ address of arr[0]
```

→ `int arr[5] = {10, 20, 30, 40, 50}`

```
int *p;
p = arr;
for (int i=0; i<arr.size(); i++){
    cout << arr " ";
    arr++;
}
```

Error

```
int arr;
p = arr;
{ 10, 20, 30, 40, 50 }
```

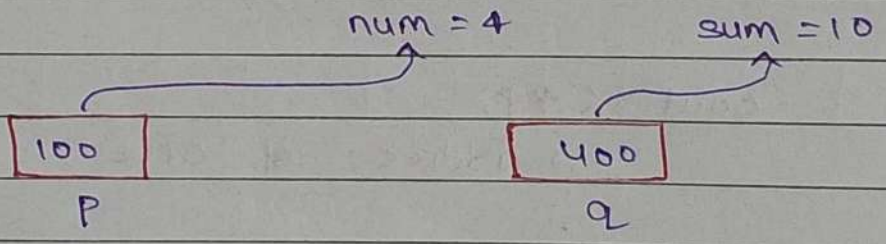
By doing p++

↓

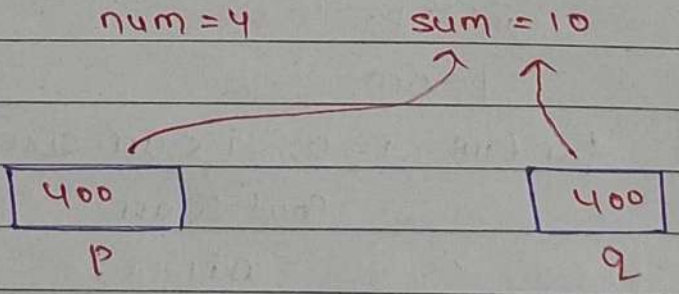
Error because it is increment the adress.

→

```
int *p, *q;
p = &num;
q = &sum;
p = q;
```



After:




```
Void fun (int c, int d) {
    c = c * 2;
    d = d * 2;
}
int main() {
    int a=5, b=8;
```

→ change c & d but in main can't reflect

Teacher's Signature.....

```
fun(a, b);                    → print 5 and 8
cout<<a<<" "<<b;
return 0;
}
```

pass by value

pass by address :

```
int main() {
    int a = 5, b = 8;
    fun(&a, &b);
    cout<<a<<" "<<b;
    return 0;
}

void fun( int *c, int *d){
    *c = *c * 2;
    *d = *d * 2;
}
```

* call by Reference :

```
void fun( int &c, int &d ){
    c = c * 2;
    d = d * 2;
}
```

Teacher's Signature.....

Page No.:
Date: / /

```
* void reverse (int * arr , int size){
        int start =0, end = size -1;
        while (start <end){
            int temp = arr [start];
            arr[start] = arr [end];
            arr[end] = temp;
            start ++;
            end --;
        }
    }

int main () {
    int a[5] = {10, 20, 30, 40, 50};
        reverse (a, 5);
        for (int i = 0; i <5; i++){
            cout << a[i] << " ";
        }
        return 0;
    }
```

Teacher's Signature.....